

Εθνικό Μετσόβιο Πολυτεχνείο

Επιστήμη Δεδομένων και Μηχανική Μάθηση

Εξαμηνιαία Εργασία

Διαχείριση Δεδομένων Μεγάλης Κλίμακας

Όνομα / Α.Μ

Νικόλαος Αναστασίου / 03400283

Αθήνα, 14 Ιουνίου 2026

Περιεχόμενα

Εισαγωγή	2
Ζητούμενο 1	2
Ζητούμενο 2 (DataFrame, χωρίς UDF)	3
Ζητούμενο 2 (DataFrame, με UDF)	3
Ζητούμενο 2 (RDD)	4
Ζητούμενο 2 Αποτελέσματα	4

Εισαγωγή

Στη παρούσα εργασία υλοποιήθηκαν τα Ζητήματα 1 έως 6 σε περιβάλλον Apache Spark και Apache Hadoop, ενώ ο κώδικας για την απάντηση των Queries γράφτηκε σε Python ακολουθώντας τα templates που μας δόθηκαν.

Η υλοποίηση έγινε τόσο σε τοπικό επίπεδο για την ανάπτυξη του κώδικα αλλά και στο απομακρυσμένο cluster που μας δόθηκε πρόσβαση μέσω VPN. Η ανάπτυξη έγινε σύμφωνα με τον οδηγό του github.com/ikons/bigdata-dsml.

Αντί να γίνει εγκατάσταση του Spark και του Hadoop στο WSL των Windows έγινε σε native Ubuntu 24.04.4 LTS καθώς αντί για Windows έχω Linux. Η μόνη διαφορά είναι η έκδοσή της Java που αντί για Java OpenJDK 11 που ήταν η προτεινόμενη χρησιμοποιήθηκε η προ εγκατεστημένη δηλαδή η Java OpenJDK 21.

Ζητούμενο 1

Τα δεδομένα που δόθηκαν για την εργασία είναι σε μορφή .csv η οποία αν και είναι εύκολη στη διαχείριση της και μπορεί να διαβαστεί ανοίγοντας το αρχείο με έναν text editor δεν είναι η βέλτιστη για την υποδομή του Spark. Για τον λόγο αυτό τα δεδομένα μετατράπηκαν στην προτεινόμενη μορφή για Spark που είναι η Parquet.

Υπάρχουν πολλές μορφές δεδομένων οι οποίες είναι καταλληλότερες για Spark έναντι του απλού .csv, αρχικά είναι η μορφή .parquet, στην οποία θα μετατραπούν τα δεδομένα της εργασίας, η οποία είναι Columnar δηλαδή αποθηκεύει τα δεδομένα ανα στήλη και όχι ανα γραμμή. Έτσι αμα ένα query ζητάει μόνο 1 στήλη από τις 10 τότε από τον δίσκο διαβάζεται μονό αυτή η μια στήλη, και με αυτό τον τρόπο πετυχαίνει μεγαλύτερες ταχύτητες read, καλύτερη συμπίεση δεδομένων, υποστηρίζει επίσης αλλαγές στη δομή των δεδομένων και Predicate Pushdown το οποίο παραλήπτει δεδομένα που δεν χρειάζονται κατά την εκτέλεση ενός query αυξάνοντας έτσι ακόμη περισσότερο την ταχύτητα εκτέλεσης.

Ακόμη ένα format κατάλληλο για Spark είναι το ORC (Optimized Row Columnar), το οποίο είναι Columnar σαν το Parquet, και έχει πολλές ομοιότητες. Αναπτύχθηκε για το Apache Hive και λειτουργεί εκεί καλύτερα από ότι στο Spark. Οι διαφορές ανάμεσα στα 2 είναι ότι: Το ORC έχει σαν μικρότερη μονάδα το Row Group ενώ το Parquet έχει ένα Page 8kb, το ORC έχει Column Pruning το οποίο προσπερνάει τα columns τα οποία δεν χρειάζονται για κάποιο query ενώ το parquet έχει column chunks τα οποία είναι ξεχωριστά αποθηκευμένα μέσα στον δίσκο. Επίσης υπάρχουν διαφορές στο φιλτράρισμα των δεδομένων καθώς στο ORC υπάρχουν φίλτρα για min/max, bloom filters και offsets ενώ στο parquet υπάρχουν αντίστοιχα για file, chunk και page.

Ζητούμενο 2 (DataFrame, χωρίς UDF)

Ξεκινώντας από το αρχείο DFQ1.py που μας δόθηκε στον οδηγό της εργασίας χρησιμοποιήθηκε σαν template. Φορτώνονται τα δεδομένα με την μέθοδο `spark.read.parquet(args.crimes_path)` αφού πρόκειται για δεδομένα της μορφής spark. Έπειτα γίνεται filter της στήλης "Premis Desc" και παρθήκαν οι τιμές ίσες με "STREET". Στη συνέχεια υπολογίζεται ο αριθμός των συνολικών Crimes για τον υπολογισμό του ποσοστού.

Πριν την εκτέλεση του κώδικα που κάνει το φιλτράρισμα για τις ώρες μέσα στην μέρα εκκίνηθηκε ένας `perf_counter` για την χρονομέτρηση των αποτελεσμάτων, έπειτα εκτελέστηκε ο κώδικας όπου για κάποιες τιμές της στήλης "TIME OCC" τις κατηγοριοποιούμε σε Πρωί, Απόγευμα, Βραδύ και Νύχτα με την μέθοδο `.withColumn`, η κατηγοριοποίηση έγινε συμφωνά με τον πίνακα 1

Έπειτα δημιουργείται μια νέα στήλη με όνομα `Day_Part` και κάνουμε `groupBy` της κατηγορίες και για κάθε κατηγορία μετρήθηκε ο αριθμός των Crimes και διαιρέθηκε με τον συνολικό αριθμό για να προκύψει το ποσοστό.

Τέλος εκτυπώνουμε τα αποτελέσματα, σταματάμε τον `perf_counter` και υπολογίζουμε τον χρόνο εκτέλεσης. Ο `perf_counter` πρέπει να σταματήσει μετά το `.show()` καθώς όλα τα προηγούμενα είναι με `lazy loading` και αρα δεν έχουν εκτελεστεί ακόμη.

Το αρχείο ανέβηκε στον server μέσω της εντολής στο listing 1, και εκτελέστηκε με την εντολή στο listing 2.

```
hdfs dfs -put -f /home/nikos/bigdata-dsml/code_nikos/Q1_DF_without_UDF.py /user/dsml00283/code_nikos/Q1_DF_without_UDF.py
```

Listing 1: Μεταφορά script στον server

```
spark-submit --conf spark.executor.cores=1 --conf spark.executor.memory=2g --conf spark.executor.instances=2 hdfs://hdfs-namenode.default.svc.cluster.local:9000/user/$DSML_USER/code_nikos/Q1_DF_without_UDF.py --crimes-path hdfs://hdfs-namenode.default.svc.cluster.local:9000/user/$DSML_USER/data_parquet/LA_Crime_Data_combined.parquet --base-path hdfs://hdfs-namenode.default.svc.cluster.local:9000/user/$DSML_USER
```

Listing 2: Υποβολή script στο Spark

Ζητούμενο 2 (DataFrame, με UDF)

Αρχικά φορτώνονται τα δεδομένα και φιλτράρονται οι τιμές της στήλης "Premis Desc" όπως και παραπάνω. Έπειτα ορίζεται η συνάρτηση `get_day_part` η οποία περνάει σαν όρισμα την τιμή `TIME_OCC` με την οποία όπως και παραπάνω κατηγοριοποιούμε τις ώρες μέσα στην ημέρα, η συνάρτηση δέχεται σαν όρισμα μια τιμή `time_occ` και επιστέφει την κατηγορία στην οποία ανήκει εκείνη η στιγμή μέσα στην μέρα, σύμφωνα με τον πίνακα 1.

Ξεκινάμε τον `perf_counter` και έπειτα με την μέθοδο `udf()` ορίζουμε την συνάρτηση σαν `udf`, και με την μέθοδο `.withColumn()` την εφαρμόζουμε στην νέα στήλη "Day_Part". Έπειτα κάνουμε `groupBy` και μετράμε τα εγκλήματα ανα κατηγορία και διαιρούμε με τα συνολικά

εγκλήματα και τέλος σταματάμε τον `perf_counter` αμέσως μετά το `.show()`. Στην συνέχεια μεταφέρθηκε το script στον server και εκτελέστηκε.

Ζητούμενο 2 (RDD)

Αρχικά φορτώνονται τα δεδομένα και φιλτράρονται οι τιμές της στήλης "Premis Desc" όπως και παραπάνω. Έπειτα ορίζετε η συνάρτηση `get_day_part` η οποία δέχεται σαν όρισμα μια γραμμή απο το DataFrame και απο αυτή την γραμμή φιλτράρετε η στήλη "TIME OCC", στη συνέχεια, όπως και παραπάνω, ανάλογα με την ώρα τη ημέρας γίνεται η κατηγοριοποίηση συμφωνά με τον πίνακα 1. Οι υλοποιήσεις εκτελέστηκαν με 2 executors (1 core, 2GB memory).

Έπειτα ξεκινάμε το `perf_counter` και κάνουμε `map` με την βοήθεια της συνάρτησης `get_day_part`. Στη πράξη η συνάρτηση θα πάει σε κάθε γραμμή του RDD και θα φτιάξει ζεύγη (key, value) που στην περίπτωση μας θα είναι π.χ (Prwi, 1).

Μετά το `map` κάνουμε `reduceByKey(lambda a, b: a + b)` το οποίο κάνει group όλα τα ίδια keys π.χ όλα τα "Prwi" μεταξύ τους, αρα θα έχουμε κάτι σαν (Prwi: 1, 1, 1, , etc,..) και έπειτα το `lambda` πάει σε κάθε group (key, value) και στο τρέχων άθροισμα `a` προσθέτει την επόμενη τιμή `b`. Αρα μετράμε πόσες τιμές έχει το κάθε group.

Στη συνέχεια κάνουμε ακόμη ενα `map` στα δεδομένα του προηγούμενου αποτελέσματος που είναι πλέον ("Prwi", 10), ("Apogeuma", 12), κτλ,.. (οι τιμές 10 και 12 είναι ενδεικτικές) και για την δεύτερη τιμή που είναι ο αριθμός των εγκλημάτων σε κάθε κατηγορία της μέρας τον διαιρούμε με τον συνολικό αριθμό εγκλημάτων για να υπολογιστεί το ποσοστό και επιπλέον κάνουμε `sort` με φθίνουσα σειρά, σταματάμε το `perf_counter` και εκτυπώνουμε τα αποτελέσματα. Η διαδικασία αυτή μπορεί να γίνει με την μέθοδο `lambda x: (x[0], x[1], round((x[1] total_street_crimes) * 100, 2))`, όπου `x` είναι όλες οι κατηγορίες ("Prwi", "Apogeuma", "Vradi", "Nixta") και `x[0]` είναι η κάθε κατηγορία απο τις 4 ενώ `x[1]` είναι η τιμή count. Για την τιμή αυτή την διαιρούμε και κρατάμε 2 δεκαδικά. Τέλος εκτελείτε άλλη μια μέθοδος `lambda x: x[2], ascending=False` όπου τώρα εκτελείτε ενα `sorting` με βάση την τιμή count κατά φθίνουσα σειρά.

Ζητούμενο 2 Αποτελέσματα

Παρακάτω παρουσιάζονται τα αποτελέσματα για την κάθε υλοποίηση όπως παρουσιάστηκαν παραπάνω. Αρχικά στον πίνακα 1 παρουσιάζονται οι κατηγορίες ανα ώρα μέρας που επιλέχθηκαν και εφαρμόστηκαν και στις 3 υλοποιήσεις.

Στη συνέχεια, στον πίνακα 2 παρουσιάζεται το αποτέλεσμα του Query 1, το οποίο είναι ίδιο και για τις 3 υλοποιήσεις. Φαίνεται το μεγαλύτερο ποσοστό εγκλημάτων στον δρόμο "STREET" να εμφανίζεται την Νύχτα με ποσοστό 43.08% ενώ το δεύτερο μεγαλύτερο το Βραδύ με ποσοστό 26.92%, δηλαδή με διάφορα 7.16% απο το πρώτο. Στη συνέχεια το Απόγευμα παρουσιάζει ποσοστό 21.23% και τέλος το Πρωί εμφανίζει ποσοστό 17.76%.

Time Start	Time End	Category
500	1159	Prwi
1200	1659	Apogeuma
1700	2059	Vradi
otherwise	-	Nixta

Πίνακας 1: Ταξινόμηση ώρας σε κατηγορίες.

Day_Part	Crime_Count	Percentage
Nixta	251094	34.08
Vradi	198292	26.92
Apogeuma	156432	21.23
Prwi	130866	17.76

Πίνακας 2: Αποτελέσματα Query 1.

Υλοποίηση	Χρόνος Εκτέλεσης (seconds)
DataFrame, without UDF	4.2034
DataFrame, with UDF	6.1014
DataFrame, RDD	10.6075
2 executors (1 core, 2GB memory)	

Πίνακας 3: Αποτελέσματα Query 1 με DataFrame με UDF, χωρίς UDF και με RDD (parquet).

Τα αποτελέσματα του πίνακα 3 μπορούν να βρεθούν και στον server με τα παρακάτω drive pod names:

- DataFrame, without UDF: q1-df-without-udf-py-98098d9e3ee1b706-driver
- DataFrame, with UDF: q1-df-with-udf-py-5dc85e9e3ee36aa7-driver
- DataFrame, RDD: q1-rdd-py-5107949e3b526971-driver

Υλοποίηση	Χρόνος Εκτέλεσης (seconds)
DataFrame, without UDF	46.2366
DataFrame, with UDF	31.7454
DataFrame, RDD	27.5870
2 executors (1 core, 2GB memory)	

Πίνακας 4: Αποτελέσματα Query 1 με DataFrame με UDF, χωρίς UDF και με RDD (csv).

Τα αποτελέσματα του πίνακα 4 μπορούν να βρεθούν και στον server με τα παρακάτω drive pod names:

- DataFrame, without UDF: q1-df-without-udf-csv-py-af46eb9e3ee9a2d6-driver
- DataFrame, with UDF: q1-df-with-udf-csv-py-581aa29e3eed3b3f-driver
- DataFrame, RDD: q1-rdd-csv-py-34b61e9e3b6f1b23-driver

Απο τα παραπάνω αποτελέσματα, στους πίνακες 3 και 4 γίνονται 2 συγκρίσεις. Αρχικά συγκρίνονται οι χρόνοι εκτέλεσης μεταξύ των διαφορετικών υλοποιήσεων στον πίνακα 3, DataFrame with UDF, DataFrame without UDF και RDD, όπου η υλοποίηση DataFrame, without UDF έχει τον μικρότερο χρόνο εκτέλεσης έπειτα είναι DataFrame, with UDF και τέλος η υλοποίηση με το RDD. Η τελευταία φαίνεται να έχει μεγαλύτερη διαφορά απο τις άλλες 2. Γενικά η υλοποίηση με UDF θα έπρεπε και στους 2 πίνακες να έχει τον μεγαλύτερο χρόνο εκτέλεσης καθώς η UDF πρέπει να περάσει απο Python σε Scala ή Java για να τρέχει και αυτό προσθέτει επιπλέον χρόνο στην εκτέλεση.

Παρόμοια αποτελέσματα βλέπουμε και τον πίνακα 4 αλλά πλέον η πιο γρήγορη υλοποίηση είναι η DataFrame, with UDF και όχι η DataFrame, without UDF όπως πριν. Επίσης η υλοποίηση RDD έχει πλέον τον μικρότερο χρόνο εκτέλεσης σε σχέση με τις άλλες 2.

Στη συνέχεια παρατηρούμε οτι οι υλοποιήσεις με .csv έχουν κατά πολύ μεγαλύτερο χρόνο εκτέλεσης σε σχέση με τις αντίστοιχες parquet. Η RDD υλοποίηση έχει χρόνο εκτέλεσης ≈ 27 sec με το csv αλλά ≈ 10 sec με την parquet. Αυτό συμβαίνει καθώς το format parquet ευνοεί κατανομημένη εργασία ενώ csv format όχι. Ακόμα μεγαλύτερες διαφορές παρατηρούμε στις υλοποιήσεις DataFrame, without UDF όπου παρουσιάζει χρόνος εκτέλεσης ≈ 46 sec με .csv ενώ μόνο ≈ 4 sec με .parquet, ομοίως για την υλοποίηση DataFrame, with UDF που έχει χρόνο εκτέλεσης ≈ 31 sec με .csv ενώ μόνο ≈ 6 sec με .parquet.

Ζητούμενο 3

Για το ζητούμενο 3 δημιουργήθηκαν 2 υλοποιήσεις, η πρώτη με DataFrame και η δεύτερη με SQL. Και τα 2 queries θα απαντήσουν στο ερώτημα ποιοι είναι οι τρεις μήνες με τα μεγαλύτερα ποσοστά εγκλημάτων κάθε έτους και θα εκτυπωθούν σε αύξουσα σειρά ως προς του μήνες και φθίνουσα ως προς τον αριθμό των εγκλημάτων και την κατάταξη τους μέσα στο κάθε έτος. Επιπλέον οι υλοποιήσεις θα τρέξουν με 2 executors (1 core, 2GB memory).

Ζητούμενο 3 (DataFrame)

Αρχικά ξεκίναμε έναν per_counter για την χρονομέτρηση και έπειτα προσθέτουμε μια στήλη στο αρχικό DataFrame με όνομα "date" και τύπου timestamp, χρησιμοποιώντας το format του χρόνου όπως φαίνεται παρακάτω. Στη συνέχεια προσθέτουμε μια στήλη year και μια στήλη month χρησιμοποιώντας τις μεθόδους "year()", "month()" και κάνουμε groupBy ανα χρόνο και ανα μήνα. Έπειτα μετρήθηκαν όλα τα εγκλήματα σε κάθε μήνα. Για κάθε χρόνο κάνουμε orderBy() σε φθίνουσα σειρά τον αριθμό εγκλημάτων με την βοήθεια της μεθόδου Window που δημιουργεί ένα "παράθυρο" για κάθε έτος. Τέλος εφαρμόζουμε ranking για κάθε γραμμή, δηλαδή για κάθε αριθμό εγκλημάτων και κρατάμε μόνο τις γραμμές με rank > 3, και εκτυπώνουμε τα αποτελέσματα και σταματάμε την χρονομέτρηση.

Ζητούμενο 3 (SQL)

Αρχικά ξεκινάει η χρονομέτρηση και δημιουργείται ένα view του DataFrame καθώς το sql query δε θα μπορεί να δει το ίδιο το DataFrame. Έπειτα γράφεται το SQL Query το οποίο χωρίζεται στα παρακάτω μέρη:

Στο πρώτο μέρος δημιουργούμε 2 στήλες μια year και μια month, με την βοήθεια των μεθόδων year και month από το view crimes. Οι συναρτήσεις year και month παίρνουν η κάθε μια τον χρόνο και τον μήνα.

Στη συνέχεια, κάνουμε groupby ανα έτος και ανα μήνα και μετράμε πόσα εγκλήματα έχουν γίνει σε κάθε μήνα και το βάζουμε στην στήλη crime_total.

Έπειτα για κάθε γραμμή στη στήλη crime_total και για κάθε έτος γίνεται ranking ανάλογα με τον αριθμό των εγκλημάτων και αποθηκεύεται στην στήλη ranking.

Ζητούμενο 3 Αποτελέσματα

year	month	crime_total	ranking
2010	1	19524	1
2010	3	18131	2
2010	7	17857	3
2011	1	18144	1
2011	7	19524	2
2011	1	17284	3
2012	1	17958	1
2012	8	17662	2
2012	5	17504	3

Πίνακας 5: Αποτελέσματα Query 2

Απο τα αποτελέσματα του πίνακα ?? παρατηρούμε ότι ο μήνας 1, δηλαδή ο Ιανουάριος έχει κάθε χρόνο τα περισσότερα εγκλήματα. Επίσης οι μήνες 7 και 8, δηλαδή οι Ιούλιος και Αύγουστος είναι μέσα στους top 3 μήνες κάθε χρόνο.

Υλοποίηση	Χρόνος Εκτέλεσης (seconds)
DataFrame	12.6401
SQL	12.3269

4 executors (1 core, 2GB memory)

Πίνακας 6: Αποτελέσματα Query 2 με DataFrame και με SQL (parquet).

Οι χρόνοι εκτέλεσης των 2 υλοποιήσεων παρουσιάζονται στον πίνακα ???. Παρατηρήθηκε ότι και για τις 2 υλοποιήσεις ο χρόνος εκτέλεσης ήταν παρόμοιος ≈ 12 sec με την SQL να είναι ελάχιστα πιο γρήγορη.

Τα αποτελέσματα του πίνακα ?? μπορούν να βρεθούν και στον server με τα παρακάτω drive pod names:

- DataFrame: q2-df-py-8733d89e3f8176d6-driver
- SQL: q2-sql-py-37840a9e3f855d6d-driver

Ζητούμενο 4

Για το ζητούμενο 4 υλοποιήθηκε κώδικας όπου υπολογίζει το μέσο ετήσιο κατακεφαλήν εισόδημα για την διετία 2020-2021. Χρησιμοποιήθηκαν τα LA_income_2021 και LA_Census_Blocks_2020 datasets. Οι υλοποιήσεις εκτελέστηκαν με 3 executors (1 cores, 2GB memory). Επιπλέον έγινε χρήση της μεθόδου explain έτσι ώστε να βρεθεί ποια στρατηγική ακολούθησε ο catalyst optimizer και στη συνέχεια πραγματοποιήθηκαν πειράματα με διαφορετικές στρατηγικές.

Ζητούμενο 4 (DataFrame)

Αρχικά φορτώνεται το income dataset και γίνονται μερικές αλλαγές όπως αλλαγή της στήλης "Zip Code" σε "ZipCode", έπειτα γίνεται ένα φιλτράρισμα, με τη μέθοδο .filter για κάθε γραμμή της στήλης "Estimated Median Income" καθώς δεν περιέχει πάντα αριθμούς αλλά σε μερικές εγγραφές έχει και λέξεις, αυτά αφαιρούνται. Έπειτα γίνεται αν υπάρχουν κόμματα, τελείες ή σύμβολα δολαρίου τότε αυτά αντικαθίστανται με το τίποτα δηλαδή με "", έτσι ώστε οι αριθμοί να είναι καθαροί.

Στη συνέχεια δημιουργούνται Schemas για το αρχείο LA_Census_Blocks_2020 καθώς το αρχείο αρχικά ήταν .geojson και όχι .csv άρα η δομή του είναι διαφορετική. Με αυτά είναι εφικτό να διαβαστεί το αρχείο και από αυτό να πάρουμε μόνο τις 2 στήλες που χρειαζόμαστε δηλαδή τις "ZCTA20" και "POP20", αν δε γίνει αυτό τότε κατά την εκτέλεση του script θα πάρουμε το σφάλμα ότι δεν φτάνει η μνήμη ram.

Έπειτα εκκινείται ο per_counter για την χρονομέτρηση και εκτελείται ένα inner join ανάμεσα στα dataframes income και census (το census έχει τα "ZCTA20" και "POP20"). Με το inner κρατάμε τις εγγραφές που έχουν και οι 2 το "Zip Code". Έτσι θα έχουμε ένα συνολικό μεγάλο dataframe όπου κάθε οικοδομικό τετράγωνο που έχει έναν "Zip Code" θα έχει και μια τιμή εισοδήματος.

Τέλος κάνουμε groupBy για τα "Zip Code" και για κάθε "Zip Code" αθροίζουμε τον πληθυσμό, έπειτα διαιρούμε το συνολικό income κάθε "Zip Code" με τον κάθε πληθυσμό του για να πάρουμε το κατακεφαλήν εισόδημα.